

# **CONMUTACION Y ENRUTAMIENTO DE REDES**

## **DOCENTE:**

Ing. Espinosa Pérez Jacob.

## **ALUMNOS:**

Hernández Velasco Luis Andrik

Carlos Alberto Sosa Perera

Ríos Cruz Roberto Javier

Martínez Miguel Alexis

Hernández Ruiz Kevin Eduardo

Damián González Braulio Antonio

## **TRABAJO:**

Reporte Patrón 'Prototype'

## **FECHA:**

10/2/2025

## **GRUPO:**

7SA

## **CARRERA:**

Ing. Sistemas Computacionales

## **INTRODUCCION**

En este reporte se explicará la implementación del patrón Prototype en el proyecto actual, así como su uso en este y aclarar algunos problemas presentados con este patrón dentro del trabajo.

## DESARROLLO

```
package DBOjetos;

import java.time.LocalDate;

public class Producto implements Cloneable {
    private int productoID;
    private String nombre;
    private String descripcion;
    private int areaID;
    private double precio;
    private int unidadesDisponibles;
    private int nivelReorden;
    private LocalDate fechaCaducidad;
    private String codigoBarras;
    private String tamañoNeto;
    private String marca;
    private String contenido;
    private String nombreArea;
    private int cantidad;
}
```

Primeramente, a la clase producto le implementamos la extensión "Cloneable", esta lo que hace que a todo objeto creado cuando se le llame con el ".clone", se creara una copia de ese producto.

```

private Producto(ProductoBuilder builder) {
    this.productoID = builder.productoID;
    this.nombre = builder.nombre;
    this.descripcion = builder.descripcion;
    this.areaID = builder.areaID;
    this.precio = builder.precio;
    this.unidadesDisponibles = builder.unidadesDisponibles;
    this.nivelReorden = builder.nivelReorden;
    this.fechaCaducidad = builder.fechaCaducidad;
    this.codigoBarras = builder.codigoBarras;
    this.tamañoNeto = builder.tamañoNeto;
    this.marca = builder.marca;
    this.contenido = builder.contenido;
    this.nombreArea = builder.nombreArea;
    this.cantidad = builder.cantidad;
}

@Override
public Producto clone() {
    try {
        return (Producto) super.clone();
    } catch (CloneNotSupportedException e) {
        throw new AssertionError(); // No debería ocurrir
    }
}

public Producto crearProductoNoEditable() {
    Producto noEditable = this.clone();
    // Configurar noEditable para que no se pueda modificar...
    return noEditable;
}

```

Seguidamente se pone un builder de toda la vida, para tener getters y setters. Pero debajo de eso, viene el primer uso de clone, primeramente, se creó un clone, el cual no será editable, y otro que sí, esto con el fin de que el usuario en caso de querer agregar un producto similar, o realizar un cambio grande, sin necesidad de arruinar le “original”, modifique el clone.

```

public int getProductoID() { return productoID; }
public String getNombre() { return nombre; }
public String getDescripcion() { return descripcion; }
public int getAreaID() { return areaID; }
public double getPrecio() { return precio; }
public int getUnidadesDisponibles() { return unidadesDisponibles; }
public int getNivelReorden() { return nivelReorden; }
public LocalDate getFechaCaducidad() { return fechaCaducidad; }
public String getCodigoBarras() { return codigoBarras; }
public String getTamañoNeto() { return tamañoNeto; }
public String getMarca() { return marca; }
public String getContenido() { return contenido; }
public String getNombreArea() { return nombreArea; }
public int getCantidad() { return cantidad; }

public void setProductoID(int productoID) { this.productoID = productoID; }
public void setNombre(String nombre) { this.nombre = nombre; }
public void setDescripcion(String descripcion) { this.descripcion = descripcion; }
public void setAreaID(int areaID) { this.areaID = areaID; }
public void setPrecio(double precio) { this.precio = precio; }
public void setUnidadesDisponibles(int unidadesDisponibles) { this.unidadesDisponibles = unidadesDisponibles; }
public void setNivelReorden(int nivelReorden) { this.nivelReorden = nivelReorden; }
public void setFechaCaducidad(LocalDate fechaCaducidad) { this.fechaCaducidad = fechaCaducidad; }
public void setCodigoBarras(String codigoBarras) { this.codigoBarras = codigoBarras; }
public void setTamañoNeto(String tamañoNeto) { this.tamañoNeto = tamañoNeto; }
public void setMarca(String marca) { this.marca = marca; }
public void setContenido(String contenido) { this.contenido = contenido; }
public void setNombreArea(String nombreArea) { this.nombreArea = nombreArea; }
public void setCantidad(int cantidad) { this.cantidad = cantidad; }

```

## Getters y Setters

```

@Override
public String toString() {
    return "Producto{" +
        "productoID=" + productoID +
        ", nombre=" + nombre + '\'' +
        ", descripcion=" + descripcion + '\'' +
        ", areaID=" + areaID +
        ", precio=" + precio +
        ", unidadesDisponibles=" + unidadesDisponibles +
        ", nivelReorden=" + nivelReorden +
        ", fechaCaducidad=" + fechaCaducidad +
        ", codigoBarras=" + codigoBarras + '\'' +
        ", tamañoNeto=" + tamañoNeto + '\'' +
        ", marca=" + marca + '\'' +
        ", contenido=" + contenido + '\'' +
        ", nombreArea=" + nombreArea + '\'' +
        ", cantidad=" + cantidad +
        '}';
}

```

Un return el cual mostrara que se regresara con que.

```

public boolean insertarProducto(Producto producto) {

    Producto editable = producto.clone();
    Producto Noeditable = producto.crearProductoNoEditable();

    boolean editableInsertado1 = insertarProductoEnBD(editable);
    boolean editableInsertado2 = insertarProductoEnBD2(Noeditable);

    return editableInsertado1 && editableInsertado2;
}

```

Para continuar, en la clase CONSULTADAO, ahí se implementa el llamado en método insertar producto, el cual crea editable y NoEditable, y manda llamar a los métodos de producto, para simplemente regresarlos.

```

private boolean insertarProductoEnBD(Producto producto) {
    String sql = "INSERT INTO productos (Nombre, Descripcion, AreaID, Precio, UnidadesDisponibles, NivelReorden, FechaCaducidad, CodigoBarras, TamanoNeto, Marca) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";

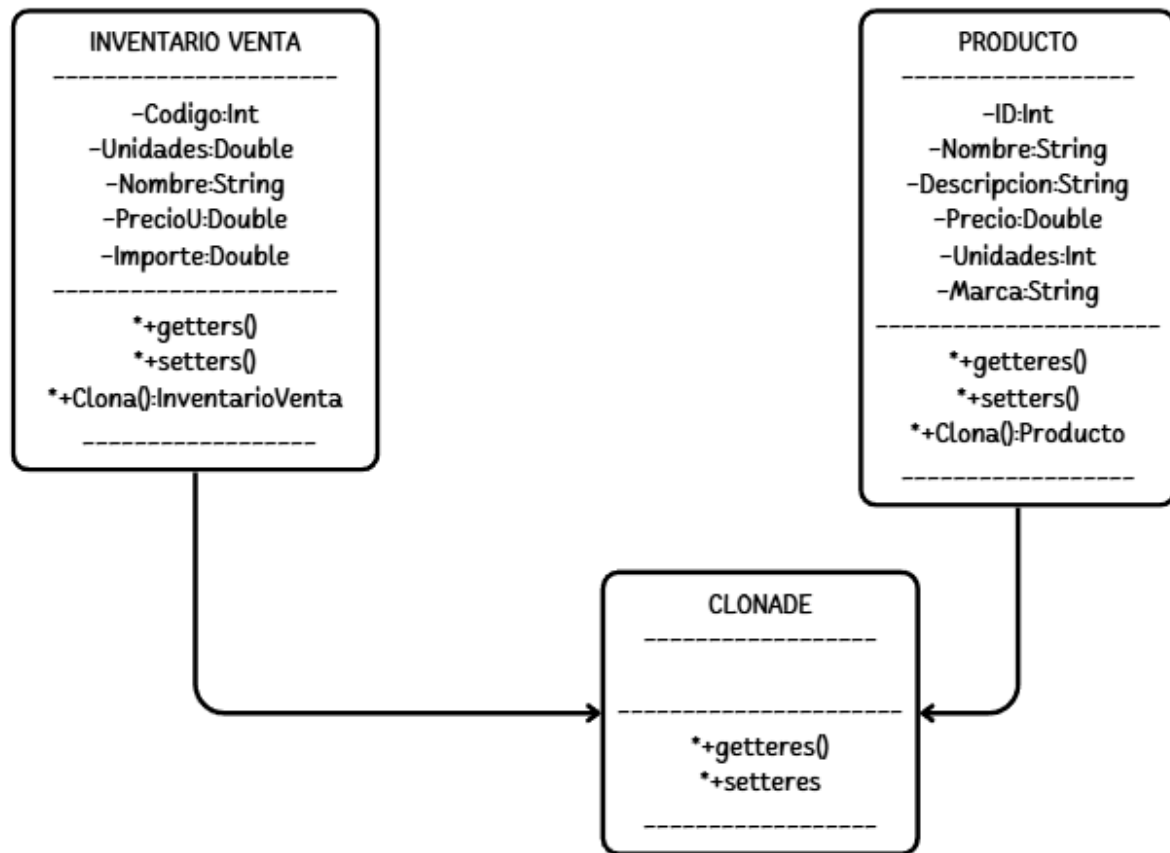
    try (PreparedStatement stmt = con.prepareStatement(sql)) {
        stmt.setString(1, producto.getNombre());
        stmt.setString(2, producto.getDescripcion());
        stmt.setInt(3, producto.getAreaID());
        stmt.setDouble(4, producto.getPrecio());
        stmt.setInt(5, producto.getUnidadesDisponibles());
        stmt.setInt(6, producto.getNivelReorden());
        stmt.setDate(7, Date.valueOf(producto.getFechaCaducidad()));
        stmt.setString(8, producto.getCodigoBarras());
        stmt.setString(9, producto.getTamanoNeto());
        stmt.setString(10, producto.getMarca());

        int affectedRows = stmt.executeUpdate();
        return affectedRows > 0;
    } catch (SQLException e) {
        LOGGER.log(Level.SEVERE, "Error al insertar producto en la base de datos", e);
        return false;
    }
}

```

Este método se ocupa en el anterior, el cual dice de qué manera se insertaran los datos en la base.

# UML



## **CONCLUSION**

En conclusión, el patrón prototype, es muy útil en la creación de objetos, por la posibilidad de la inmutabilidad y la clonación de estos, así como que se puede usar en control de errores.